S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 05

Aim: Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively.

Name: Yutika Manekar

USN: CM24061

Semester / Year:

Academic Session:

Date of Performance:

Date of Submission:

- **Aim:** Write a program to implement Shortest Job First (SJF) Preemptive Scheduling for three processes and calculate the total context switches and average waiting time. The processes have burst times 10ns, 20ns, and 30ns, arriving at 0ns, 2ns, and 6ns, respectively.

✞ **Objectives:**

Understand SJF Preemptive Scheduling: Implement the **Shortest Job First (SJF) Preemptive Scheduling** algorithm to manage CPU execution efficiently.

Calculate Context Switches: Determine the total number of context switches required for the given set of processes.

Evaluate Waiting Time: Compute the **average waiting time** for all processes before getting CPU execution.

✞ **Requirements:**

 Hardware Requirements:

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

 Software Requirements:

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

✞ **Theory:**

CPU Scheduling in Operating Systems

Introduction

Scheduling is the method by which processes are given access to the CPU. Efficient scheduling is essential for optimal system performance and user experience. There are two primary types of CPU scheduling: **Preemptive Scheduling** and **Non-Preemptive Scheduling**.

Understanding the differences between these scheduling types helps in designing and choosing the right scheduling algorithms for different operating systems.

---

1. Preemptive Scheduling

In **Preemptive Scheduling**, the operating system can interrupt or preempt a running process to allocate CPU time to another process, typically based on

priority or time-sharing policies. A process can be switched from the **running state to the ready state** at any time. Algorithms Based on Preemptive Scheduling:

- **Round Robin (RR)**
- **Shortest Remaining Time First (SRTF)**
- **Priority Scheduling (Preemptive version) Example:**

In the following case, **P2 is preempted at time 1** due to the arrival of a higher- priority process.

Advantages of Preemptive Scheduling:

✓ Prevents a process from monopolizing the CPU, improving system reliability.

✓ Enhances **average response time**, making it beneficial for multi- programming environments.

✓ Used in modern operating systems like **Windows, Linux, and macOS**.

Disadvantages of Preemptive Scheduling: More complex to implement.

Involves **overhead** for suspending a running process and switching contexts.

May cause starvation if low-priority processes are frequently preempted.

Can create **concurrency issues**, especially when accessing shared resources.

---

2. **Non-Preemptive Scheduling**

In **Non-Preemptive Scheduling**, a running process cannot be interrupted by the operating system. It continues executing until it **terminates** or **enters a waiting state** voluntarily.

Algorithms Based on Non-Preemptive Scheduling:

- **First Come First Serve (FCFS)**
- **Shortest Job First (SJF - Non-Preemptive)**
- **Priority Scheduling (Non-Preemptive version) Example:**

Below is a **Gantt Chart** based on the **FCFS algorithm**, where each process executes fully before the next one starts.

Advantages of Non-Preemptive Scheduling:

✓ **Easy to implement** in an operating system (used in older versions like Windows 3.11 and early macOS).

✓ **Minimal scheduling overhead** due to fewer context switches.

✓ **Less computational resource usage**, making it more efficient for simpler systems.

Disadvantages of Non-Preemptive Scheduling:

Risk of Denial of Service (DoS) attacks, as a process can monopolize the CPU.

Poor response time, especially in multi-user systems.

---

3. **Differences Between Preemptive and Non-Preemptive Scheduling**

Parameter	Preemptive Scheduling	Non-Preemptive Scheduling
Basic Concept	CPU time is allocated for a **limited time**.	CPU is held until process **terminates** or enters waiting state.
Interrupts	Process **can be interrupted**.	Process **cannot be interrupted**.
Starvation	Frequent high-priority processes may starve low-priority ones.	A long-running process can starve later-arriving shorter processes.
Overhead	Higher overhead due to frequent **context switching**.	Minimal overhead.
Flexibility	More flexible (critical processes get priority).	Rigid scheduling approach.
Response Time	Faster response time.	Slower response time.
Process Control	**OS has more control** over scheduling.	OS has **less control** over scheduling.
Concurrency Issues	**Higher**, as processes may be preempted during shared resource access.	**Lower**, as processes run to completion.
Examples	Round Robin, SRTF.	FCFS, Non-Preemptive SJF.

---

4. **Frequently Asked Questions (FAQs)**

a. How is priority determined in Preemptive scheduling?

Ans: Preemptive scheduling systems assign priority based on **task importance, deadlines, or urgency**. Higher-priority tasks execute before lower-priority ones.

b. What happens in non-preemptive scheduling if a process does not yield the CPU?

Ans: If a process does not voluntarily yield the CPU, it can lead to **starvation or deadlock**, where other tasks are unable to execute.

c. Which scheduling method is better for real-time systems?

Ans: Preemptive scheduling is better for **real-time systems**, as it allows high- priority tasks to execute immediately.

✞ **CODE**

sjf_preemptive.c :

```
class Process:     def __init__(self, pid,
arrival, burst):
        self.pid = pid
self.arrival = arrival
self.burst = burst
self.remaining = burst
        self.start = -1
self.finish = 0
self.waiting = 0
        self.completed = False

def find_shortest_job(processes, current_time):
    shortest = None
    min_remaining = float('inf')

    for p in processes:        if p.arrival <= current_time and not p.completed
and p.remaining > 0:            if p.remaining < min_remaining:
min_remaining = p.remaining                shortest = p
    return shortest

def main():
    processes = [
Process(1, 0, 10),
        Process(2, 2, 20),
        Process(3, 6, 30)
    ]
    print("Shortest Job First (Preemptive) Scheduling")
print("=" * 50)
    print("Process | Arrival | Burst")
print("--------|---------|------")     for p
in processes:
        print(f"P{p.pid}      | {p.arrival:2d} ns   | {p.burst:2d} ns")

    current_time = 0
completed = 0     current_pid
= -1
    context_switches = -1
```

```
    print("\nGantt Chart:")
print("Time  | Process")
    print("------|--------")

    while completed < len(processes):
        shortest = find_shortest_job(processes, current_time)
if shortest:            if current_pid != shortest.pid:
context_switches += 1                current_pid = shortest.pid
if shortest.start == -1:
                shortest.start = current_time
            print(f"{current_time:3d} ns | P{shortest.pid}")

          shortest.remaining -= 1

          if shortest.remaining == 0:
shortest.completed = True
shortest.finish = current_time + 1
completed += 1        else:            if
current_pid != -2:
context_switches += 1
            print(f"{current_time:3d} ns | CPU Idle")
            current_pid = -2

        # Update waiting times
for p in processes:
          if (p.arrival <= current_time and not p.completed and
p != shortest and p.remaining > 0):                p.waiting += 1

        current_time += 1

    # Calculate turnaround times
for p in processes:
        p.turnaround = p.finish - p.arrival
total_waiting = sum(p.waiting for p in processes)
    avg_waiting = total_waiting / len(processes)

    print("\nProcess Details:")
    print("PID | Arrival | Burst | Start | Finish | Waiting | Turnaround")
print("----|---------|-------|-------|--------|---------|-----------")     for p in
processes:
        print(f"P{p.pid}  | {p.arrival:2d} ns   | {p.burst:2d} ns | "
            f"{p.start:2d} ns | {p.finish:2d} ns   | "
            f"{p.waiting:2d} ns    | {p.turnaround:2d} ns")
```

```
    print(f"\nResults:")
    print(f"Total Context Switches: {context_switches}")
print(f"Total Waiting Time: {total_waiting:.2f} ns")
    print(f"Average Waiting Time: {avg_waiting:.2f} ns")

if __name__ == "__main__":
    main()
```

✞ **Output:**

```
ubuntu@ubuntu:~$ nano sjf_preemptive.c
ubuntu@ubuntu:~$ gcc -o sjf_preemptive sjf_preemptive.c
ubuntu@ubuntu:~$ ./sjf_preemptive
Shortest Job First (Preemptive) Scheduling Simulation
=====================================================
Process | Arrival | Burst
--------|---------|------
P1      |  0 ns   | 10 ns
P2      |  2 ns   | 20 ns
P3      |  6 ns   | 30 ns

Gantt Chart:
Time   | Process
------|--------
  0 ns | P1
 10 ns | P2
 30 ns | P3

Process Details:
PID | Arrival | Burst | Start | Finish | Waiting | Turnaround
----|---------|-------|-------|--------|---------|-----------
P1  |  0 ns   | 10 ns |  0 ns | 10 ns  |  0 ns   | 10 ns
P2  |  2 ns   | 20 ns | 10 ns | 30 ns  |  8 ns   | 28 ns
P3  |  6 ns   | 30 ns | 30 ns | 60 ns  | 24 ns   | 54 ns

Results:
========
Total Context Switches: 2
Total Waiting Time: 32.00 ns
Average Waiting Time: 10.67 ns

Explanation:
============
1. At time 0ns, only P1 is available, so it starts execution.
2. At time 2ns, P2 arrives. P1 has 8ns remaining, P2 needs 20ns.
   P1 continues as it has shorter remaining time.
3. At time 6ns, P3 arrives. Now we compare remaining times:
   P1: 4ns remaining, P2: 20ns remaining, P3: 30ns
   P1 still has shortest remaining time, so it continues.
4. P1 completes at time 10ns.
5. Now at time 10ns, compare P2 (20ns) and P3 (30ns).
   P2 has shorter burst, so it executes.
6. P2 continues until completion at time 30ns.
7. P3 executes from time 30ns to 60ns.
ubuntu@ubuntu:~$
```

Conclusion: Preemptive scheduling offers better responsiveness but adds complexity, while non-preemptive scheduling is simpler but may cause inefficiencies. The choice depends on system needs, with preemptive suited for multitasking and non-preemptive for low-overhead scenarios.

✞ **Discussion Questions:**

1. **What is the key difference between preemptive and non-preemptive scheduling?**
2. **Why does preemptive scheduling require context switching?**
3. **Which CPU scheduling algorithm is most suitable for real-time systems and why?**
4. **What is starvation in CPU scheduling, and how can it be prevented?**
5. **Why is the Round Robin scheduling algorithm preferred in time-sharing systems?**

✞ **References:**

https://www.geeksforgeeks.org/preemptive-and-non-preemptive-scheduling/

Date: / /2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26